CSC 340-03

Programming Methodology
Week 5

- Review Week 4

Instructor: Joseph Hui

Email: jhui@sfsu.edu

A Link-Based Implementation of the ADT Bag

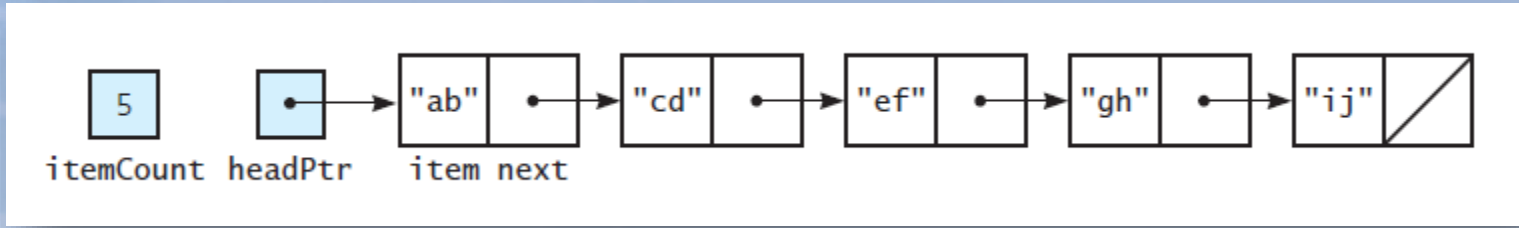


FIGURE 4-6 A link-based implementation of the ADT bag

```
+getCurrentSize(): integer
+isEmpty(): boolean
+add(newEntry: ItemType): boolean
+remove(anEntry: ItemType): boolean
+clear(): void
+getFrequencyOf(anEntry: ItemType): integer
+contains(anEntry: ItemType): boolean
+toVector(): vector
```

Bag operations, given in UML notation

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Defining the Core Methods

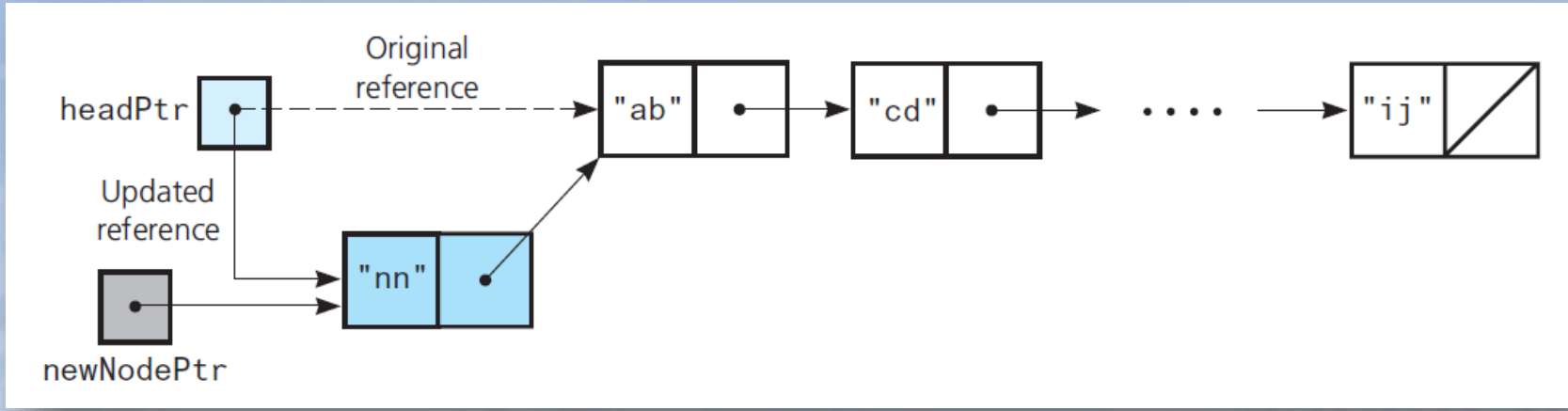


FIGURE 4-7 Inserting at the beginning of a linked chain

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Defining the Core Methods

- Traverse operation visits each node in linked chain
  - Must move from node to node

```
Let a current pointer point to the first node in the chain
while (the current pointer is not the null pointer)
{
    Assign the data portion of the current node to the next element in a vector
    Set the current pointer to the next pointer of the current node
}
```

High-level pseudocode for this loop

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Defining the Core Methods

```
template<class ItemType>
std::vector<ItemType> LinkedBag<ItemType>::toVector() const
{
   std::vector<ItemType> bagContents;
   Node<ItemType>* curPtr = headPtr;
   int counter = 0;
   while ((curPtr != nullptr) && (counter < itemCount))
   {
      bagContents.push_back(curPtr->getItem());
      curPtr = curPtr->getNext();
      counter++;
   }  // end while
   return bagContents;
}  // end toVector
```

Definition of toVector

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing More Methods

```
template<class ItemType>
bool LinkedBag<ItemType>::remove(const ItemType& anEntry)
{
   Node<ItemType>* entryNodePtr = getPointerTo(anEntry);
   bool canRemoveItem = !isEmpty() && (entryNodePtr != nullptr);
   if (canRemoveItem)
   {
      // Copy data from first node to located node
      entryNodePtr->setItem(headPtr->getItem());

      // Disconnect first node
      Node<ItemType>* nodeToDeletePtr = headPtr;
      headPtr = headPtr->getNext();

      // Return node to the system
      nodeToDeletePtr->setNext(nullptr);
      delete nodeToDeletePtr;
      nodeToDeletePtr = nullptr;

      itemCount--;
   }  // end if

   return canRemoveItem;
}  // end remove
```

Method remove also calls getPointerTo

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Recursive Definitions Methods in LinkedBag

- Private method getPointerTo
  - Locates given entry within linked chain
  - Traversal stops if it locates node that contains given entry

```
template<class ItemType>
Node<ItemType>* LinkedBag<ItemType>::getPointerTo(const ItemType& target,
                                                  Node<ItemType>* curPtr) const
{
   Node<ItemType>* result = nullptr;
   if (curPtr != nullptr)
   {
      if (target == curPtr->getItem())
         result = curPtr;
      else
         result = getPointerTo(target, curPtr->getNext());
   }  // end if

   return result;
}  // end getPointerTo
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Array-Based vs. and Link-Based

- Arrays easy to use, but have fixed size
  - Not always easy to predict number of items in ADT
  - Array could waste space
  - Increasing size of dynamically allocated array can waste storage *and* time
  - Can access array items directly with equal access time
  - An array-based implementation is a good choice for a small bag
- Linked chains do not have fixed size
  - In a chain of linked nodes, an item points explicitly to the next item
  - Link-based implementation requires more memory
  - Must traverse a linked chain to access its ith node
  - Time to access i^{th} node in a linked chain depends on i

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Dictionary

```
Dictionary
---------------------------------------------------------------
---------------------------------------------------------------
+isEmpty(): boolean
+getNumberOfEntries(): integer
+add(searchKey: KeyType, newValue: ValueType): boolean
+remove(targetKey: KeyType): boolean
+clear(): void
+getValue(targetKey: KeyType): ValueType
+contains(targetKey: KeyType): boolean
+traverse(visit(value: ValueType): void): void
```

FIGURE 18-2 UML diagram for a class of dictionaries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

- Categories of linear implementations
  - Sorted by search key array-based
  - Sorted by search key link-based

  insert a new item into proper position based on the value of search key

  - Unsorted array-based
  - Unsorted link-based

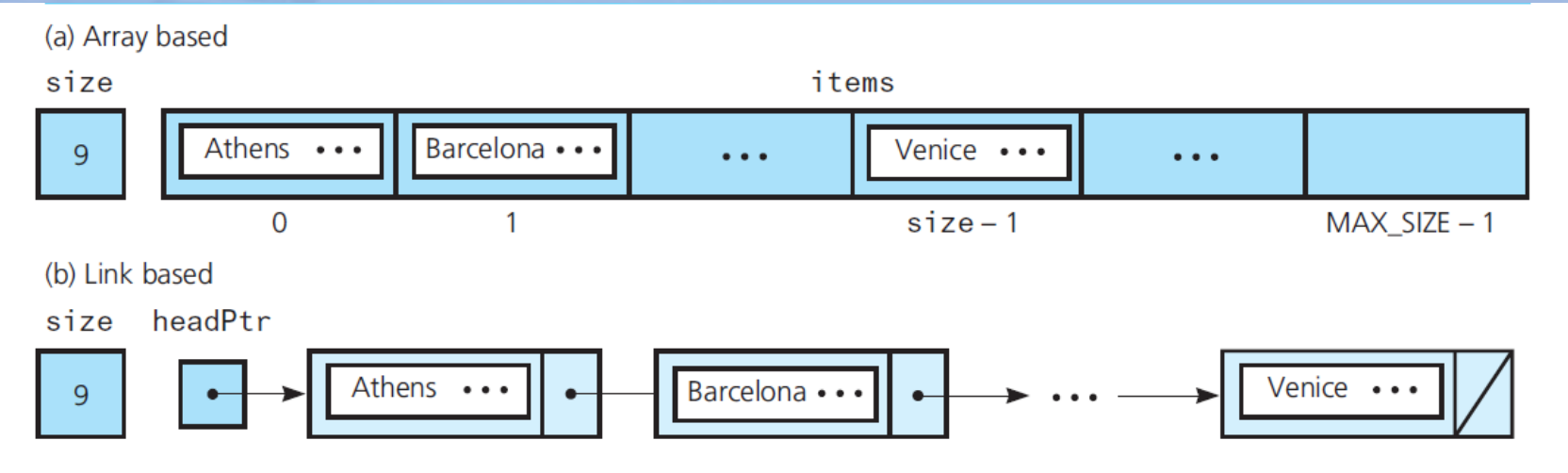
  insert a new item into any convenient location

Search key	Data Value

FIGURE 18-3 A dictionary entry

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations



- A linear array-based implementation of the ADT dictionary must shift data during an insertion or deletion in sorted order, which can be expensive, particularly for large dictionaries.
- A linear link-based implementation of the ADT dictionary eliminates the need to shift data, it does not support the insertion or removal operations any more efficiently than an array-based implementation, because you cannot perform a binary search in a reasonable fashion.

Figure 18-4 Data members for two sorted linear implementations of the ADT dictionary for the data in Figure 18-1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Possible Implementations

Another option is implementing it as a binary search tree.

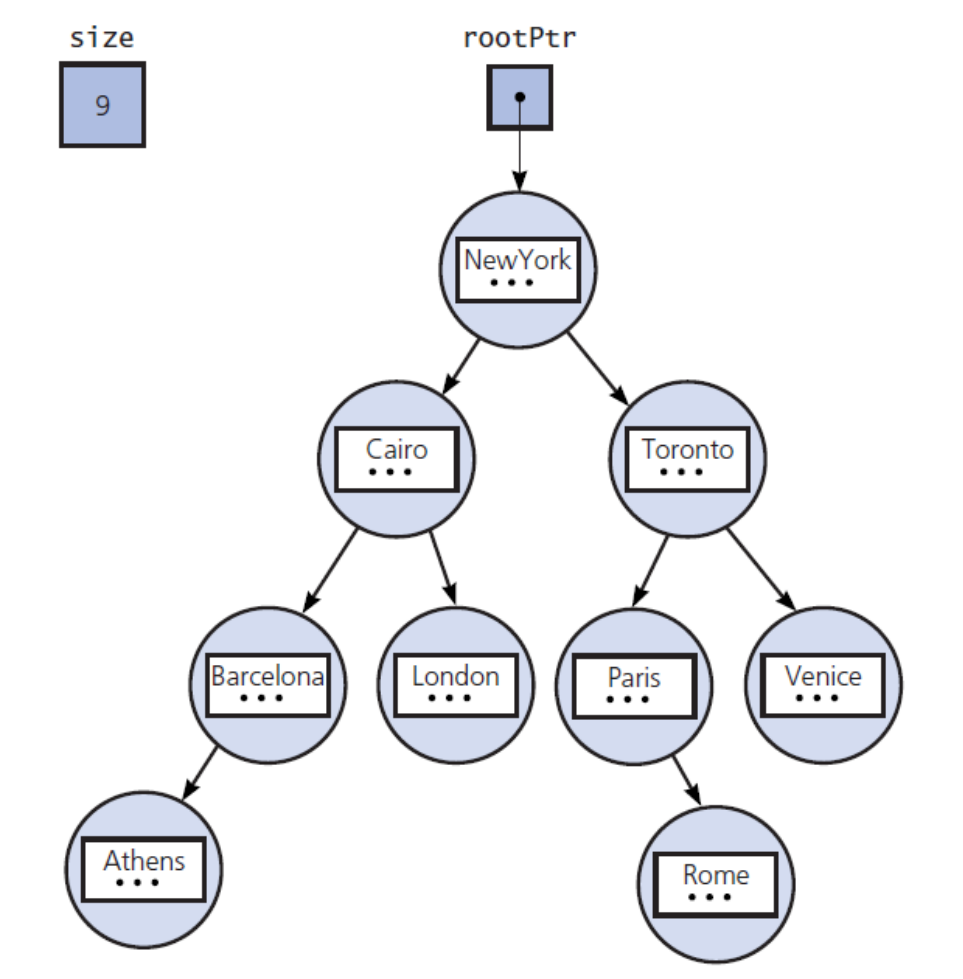


FIGURE 18-5 The data members for a binary search tree implementation of the ADT dictionary for the data in Figure 18-1

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

A. Addition and traversal in no particular order
   - Unsorted order is efficient
   - Array-based versus pointer-based

B. Retrieval
   - Sorted array-based can use binary search
   - Binary search impractical for link-based
   - Max size of dictionary affects choice

C. Addition, removal, retrieval, traversal in sorted order
   - Add and remove need to find position, then add or remove from that position
   - Array-based best for find, link-based best for addition/removal

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

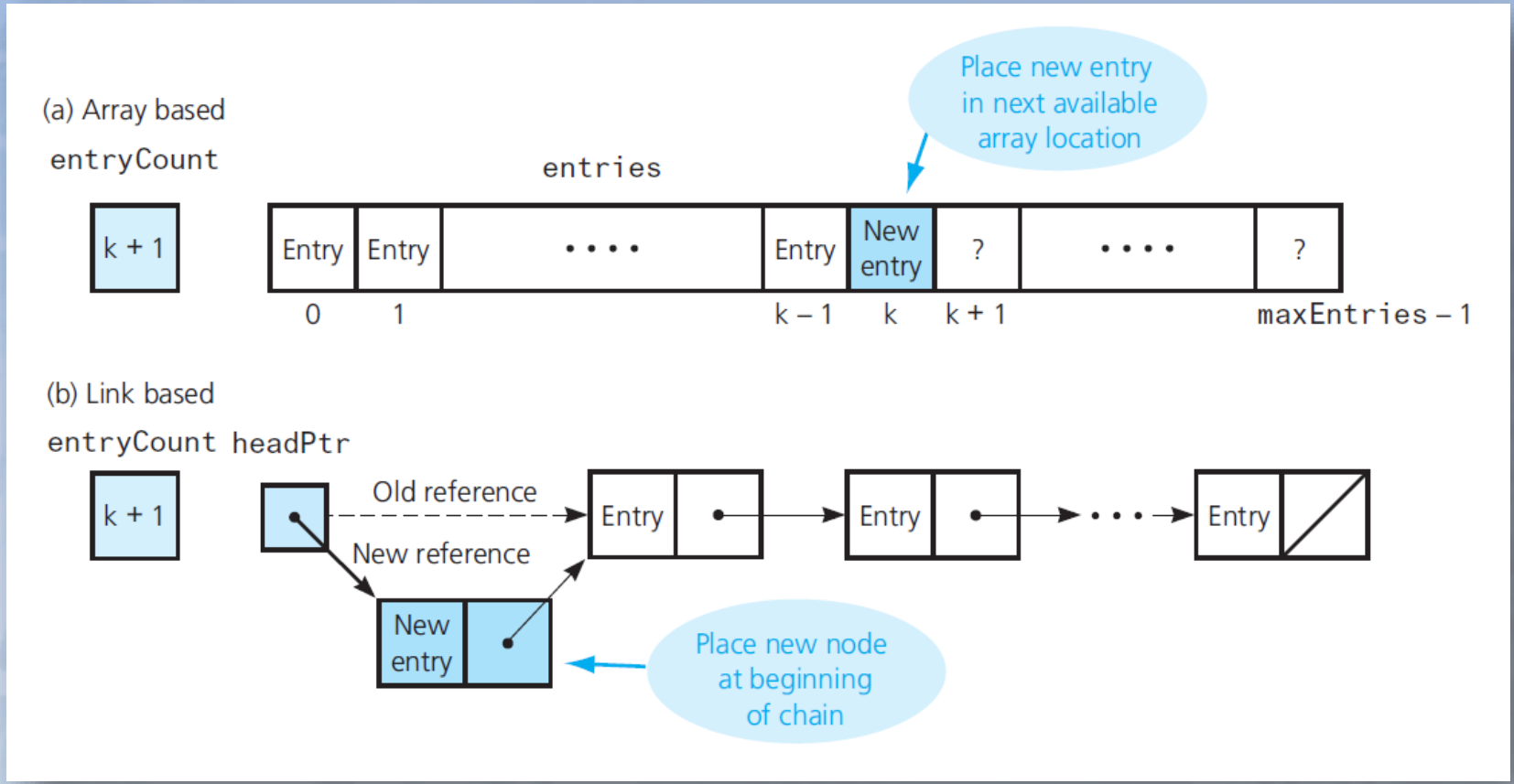


FIGURE 18-6 Addition for unsorted linear implementations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Three Scenarios

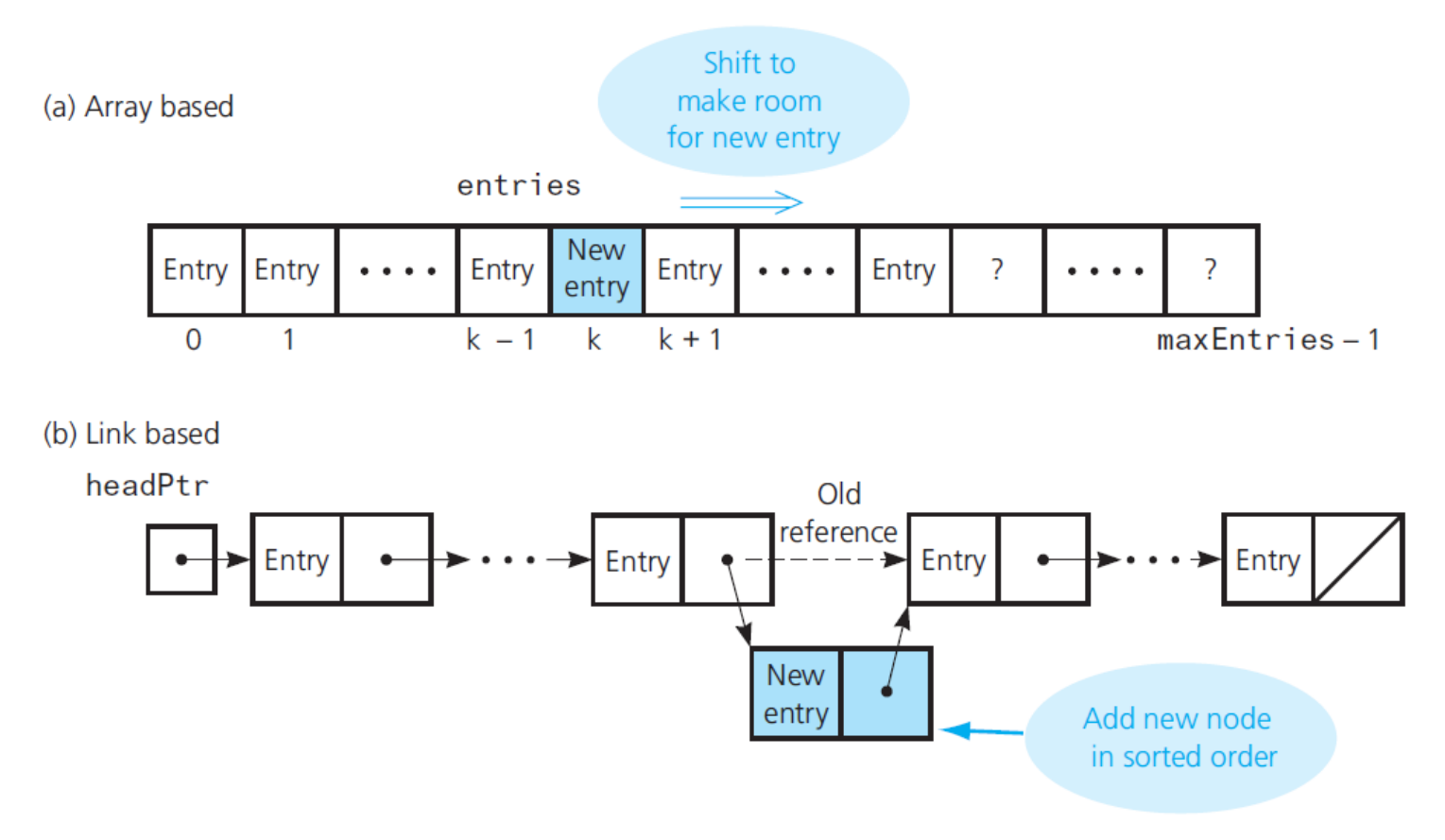


FIGURE 18-7 Addition for sorted linear implementations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Situations occur for which search-tree implementations are not adequate.
- Consider a method which acts as an “address calculator” which determines an array index
  - Used for add, getValue, remove operations
- Called a hash function
  - Tells where to place item in a hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

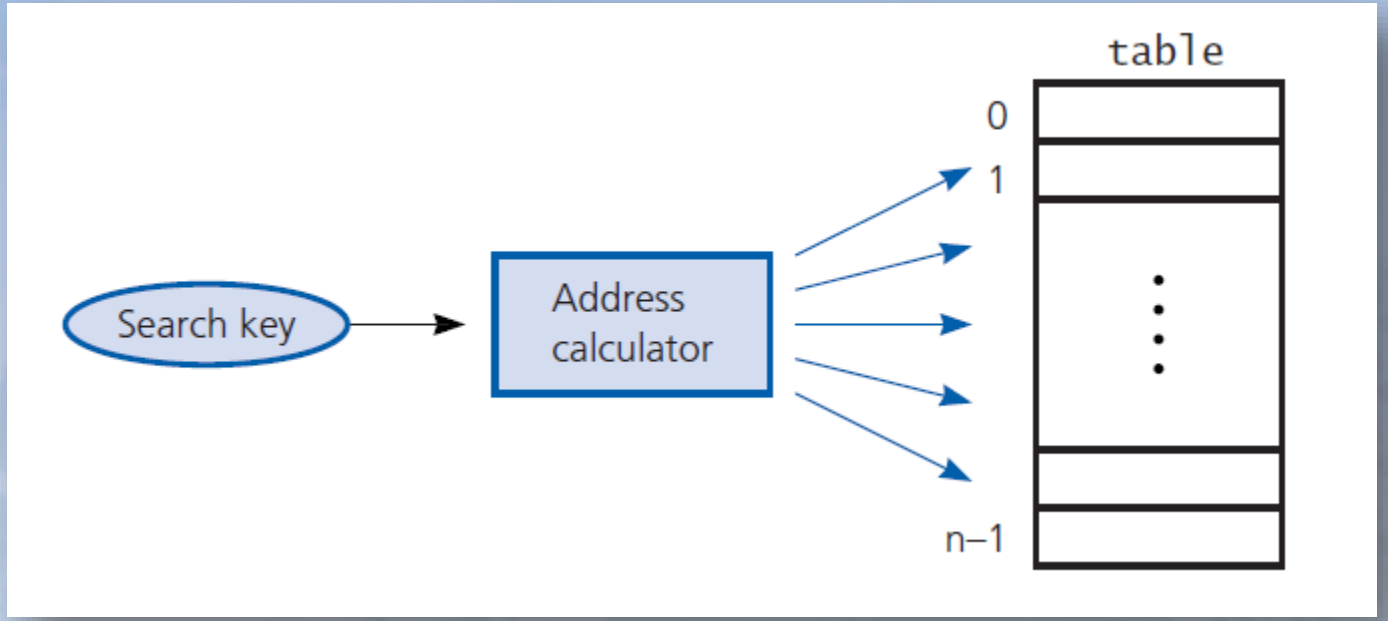


FIGURE 18-9 Address calculator

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hashing as a Dictionary Implementation

- Perfect hash function
  - Maps each search key into a unique location of the hash table
  - Possible if you know all the search keys
- Collision occurs when hash function maps more than one entry into same array location
- Hash function should
  - Be easy, fast to compute
  - Place entries evenly throughout hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Hash Functions

- Sufficient for hash functions to operate on integers – examples:
  - Select digits from an ID number
    - Generally uneven distribution
  - Folding – add digits, sum is the table location
  - Modulo arithmetic $h(x) = x \bmod tableSize$
    - Simple, effective
    - Use prime number for table size
  - Convert character string to an integer – use ASCII values
    - Address potential overflow

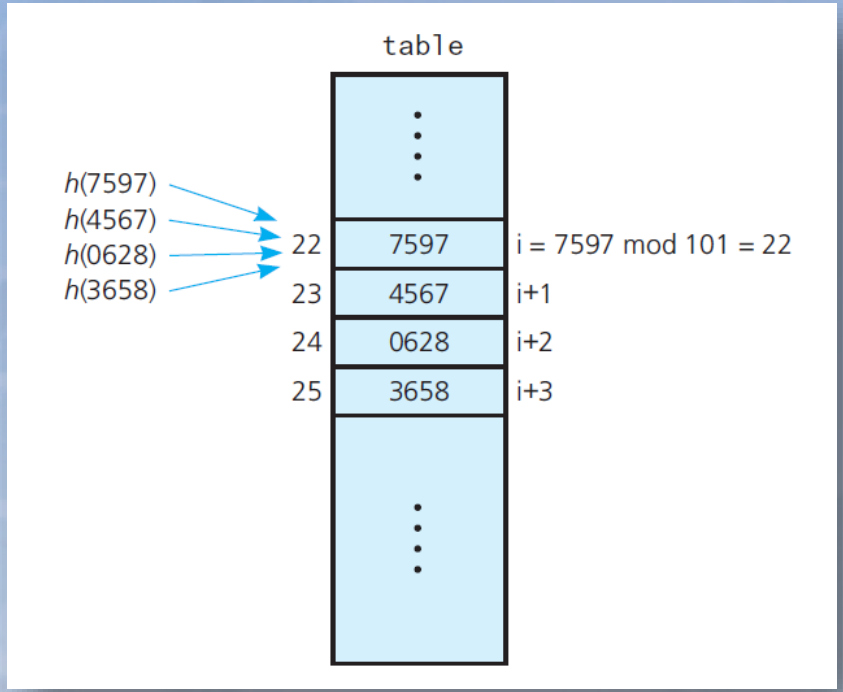
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing

- Approach 1: Open addressing
  - Linear probing
  - Quadratic probing
  - Double hashing
  - Increase size of hash table

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



Linear probing

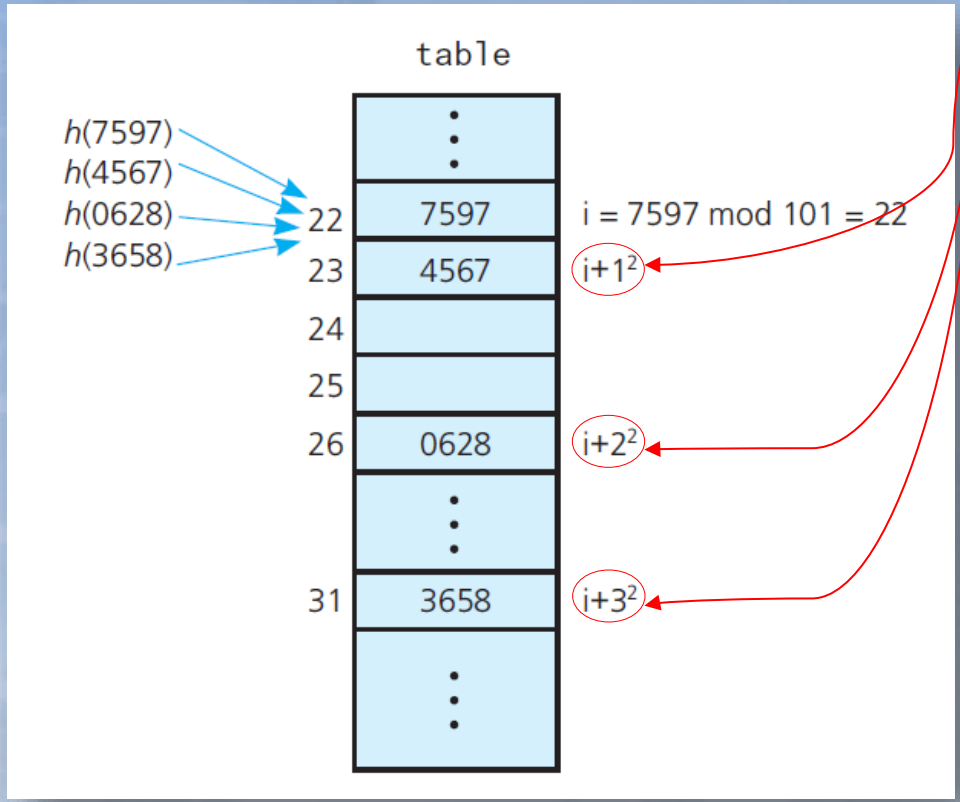
- Add - search the hash table, starting from the original hash location; if table[h(searchKey)] is occupied, check table[h(searchKey)+1] , table[h(searchKey)+2], … until you find an available location.
- Delete –place table location into one of three states:
  - occupied (currently in use),
  - empty (has not been used), or
    - Modify addItem to insert at this location
  - removed (was once occupied but is now available)
    - Modify getItem to continue probing
    - Modify addItem to insert at this location

- Items tend to cluster together in the hash table, when using this approach - primary clustering.
- Clusters can get close to one another and, merge into a larger cluster.
- Large clusters tend to get even larger

FIGURE 18-11 Linear probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



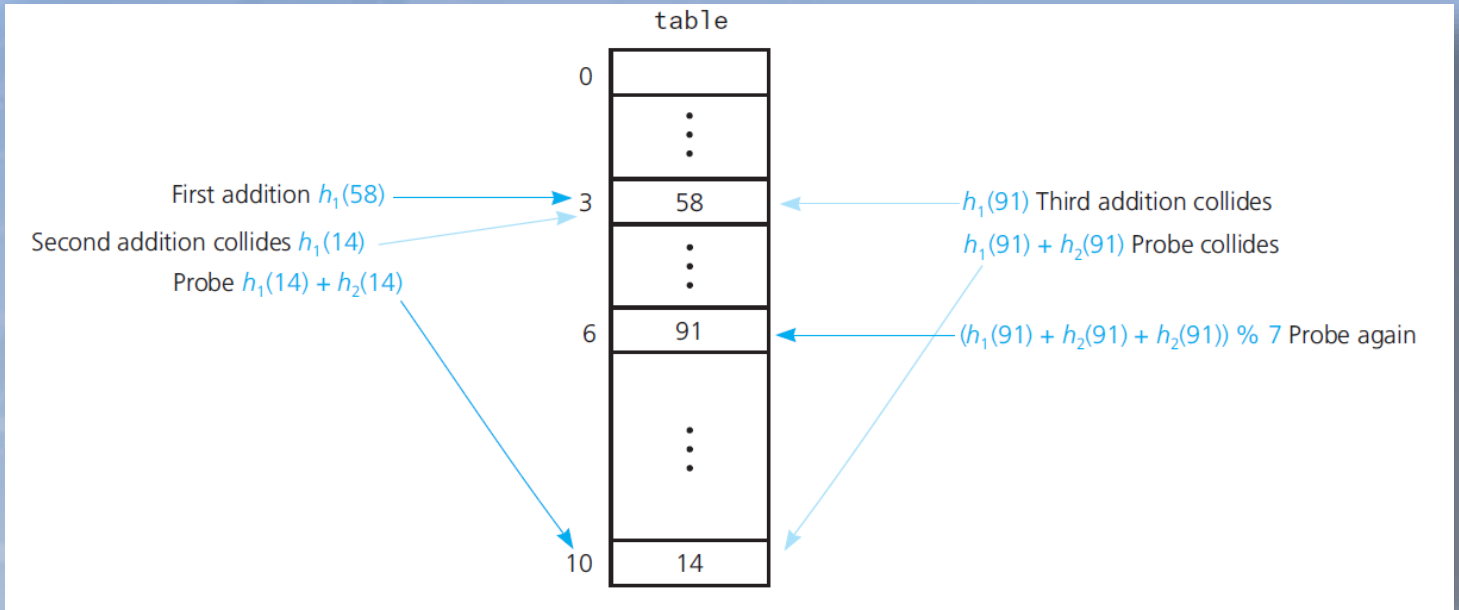
Quadratic probing.

- You can virtually eliminate primary clusters by adjusting the linear probing scheme.
- When two items hash collide, quadratic probing uses the same probe sequence for each item.
- The resulting phenomenon—called secondary clustering —delays the resolution of the collision.
- Although the research of quadratic probing remains incomplete, it appears that secondary clustering is not a problem.

FIGURE 18-12 Quadratic probing with h(x) = x mod 101

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions with Open Addressing



If key = 58,

- h1 hashes to location 3 (58 mod 11), and
- h2 indicates that the probe sequence should take steps of size 5 (7 – 58 mod 7).
- In other words, the probe sequence will be 3, 8, 2 (wraps around), 7, 1 (wraps around), 6, 0, 5, 10, 4, 9.

Double hashing - defines key-dependent probe sequences.

- searches the hash table in a linear order, starting at the location h1(key),
- second hash function h2 determines the size of the steps taken
  - h2(key) <> 0
  - h2 <> h1
- For example, let h1 and h2 be the primary and secondary hash functions defined as
  - h1 (key) = key mod 11 // 11 is table size and prime number
  - h2(key) = 7 - (key mod 7) // visits all the table locations (table size and step are prime)

FIGURE 18-13 Double hashing during the addition of 58, 14, and 91

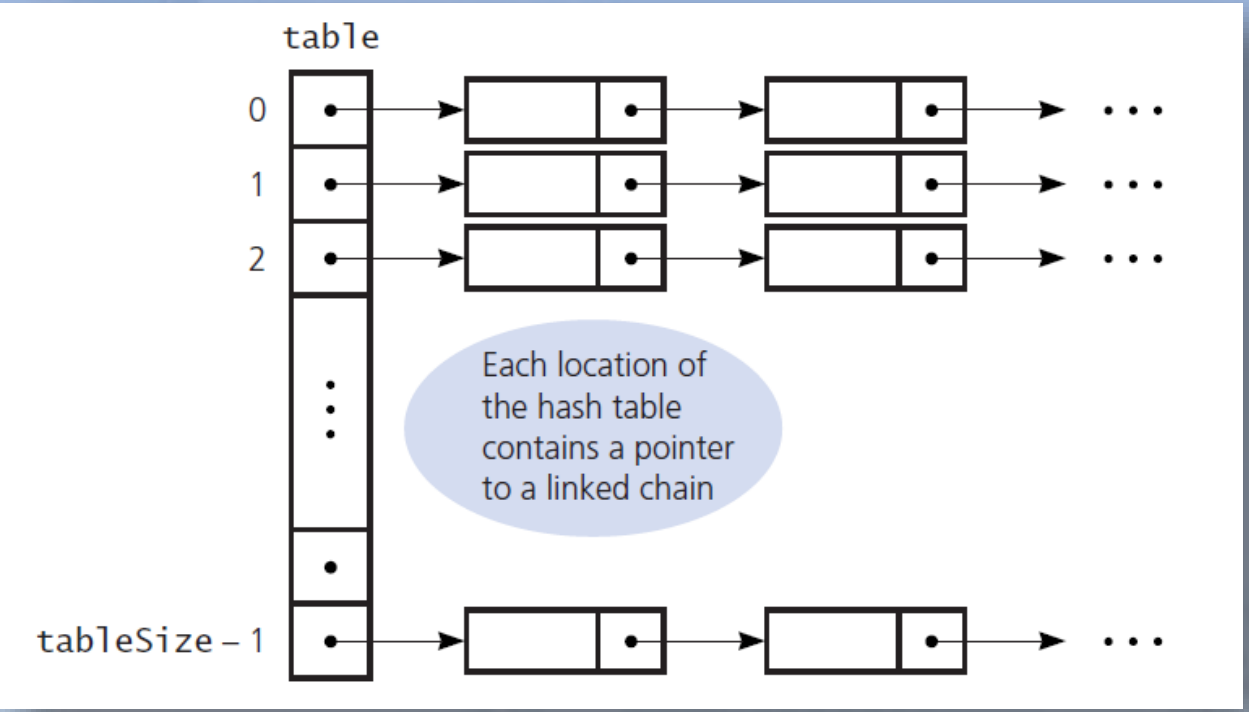
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions - Table Restructure

- You should not double the size of the array, since the size of the hash table should be prime.
- You do not move the items from the original hash table to the new hash table, since your hash function may have been impacted, e.g., x mod tableSize
- You need to apply your new hash function to every item in the old hash table before placing it into the new hash table. (Rehashing); expensive
- Approach 2: Resolving collisions by restructuring the hash table
  - Buckets
    - Define each has table location table[i] as an array, aka, bucket, where you store the collided items into in this array
    - Too small – only postponed collision problem
    - Too big – wasted storage
  - Separate chaining

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Resolving Collisions - Table Restructure



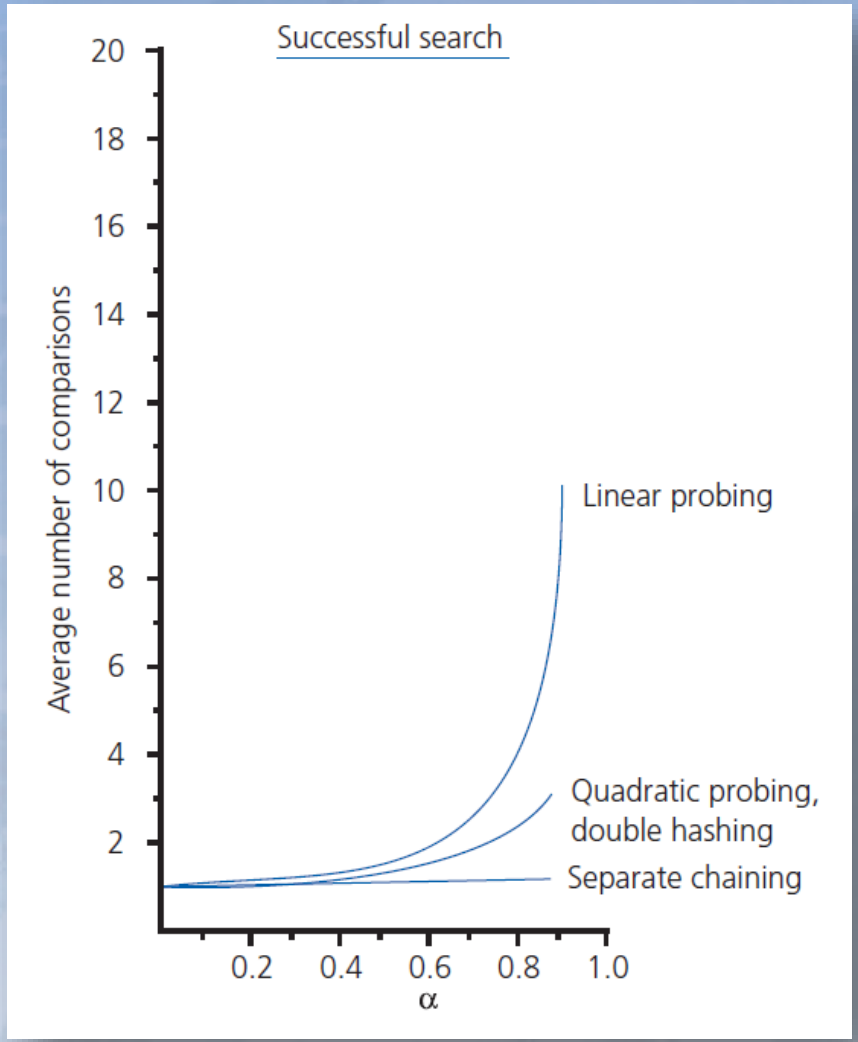
Separate chaining. Design the hash table as an array of linked chains

- Each entry table[i] is a pointer to a chain of linked nodes containing the items that the hash function has mapped into location i
- The size of the dictionary is dynamic and can exceed the size of the hash table, because each linked chain can be as long as necessary.
- The most time-efficient collision-resolution scheme.

FIGURE 18-14 Separate chaining

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Efficiency of Hashing



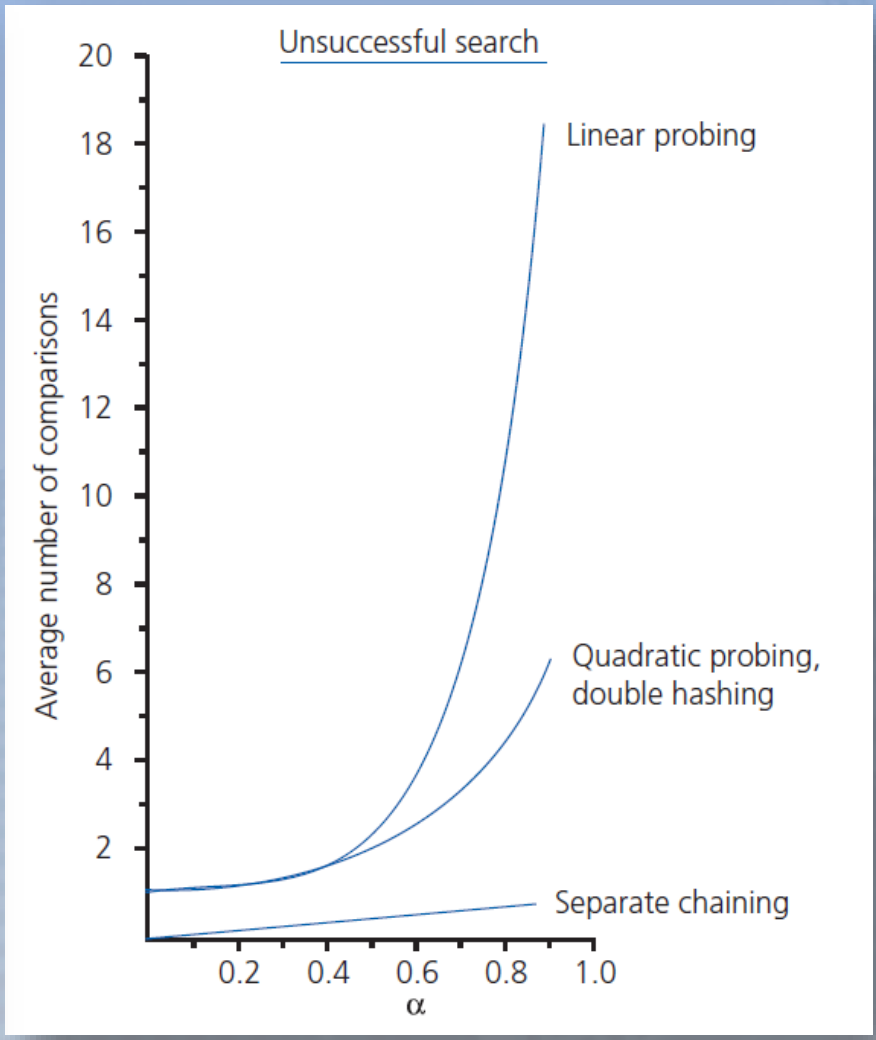


FIGURE 18-15 The relative efficiency of four collision-resolution methods

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

What Constitutes a Good Hash Function?

- Is hash function easy and fast to compute?
- Does hash function scatter data evenly throughout hash table?
- How well does hash function scatter random data?
- How well does hash function scatter *non*-random data?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Dictionary Traversal: An Inefficient Operation Under Hashing

- Entries hashed into table[i] and table[i+1] have no ordering relationship
- Hashing does not support well traversing a dictionary in sorted order
  - Generally better to use a search tree
- In external storage possible to see
  - Hashing implementation of getValue
  - And search-tree for ordered operations simultaneously

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End